

Vibe Coding 101 with Replit

Principles of Agentic Code Development

Be Precise

- Give Replit agent one task at a time
 - Break down complex tasks into smaller pieces
 - Handling multiple complex tasks can be challenging for today's coding agents
- Make your prompts specific and detailed

Be Well-Organized

- Keep your project tidy
 - Add features step-by-step
 - Test features after each addition
 - Roll back when something breaks
- Start with a fresh session for each new feature – easier to rollback
- Do not hesitate to return to the last working version

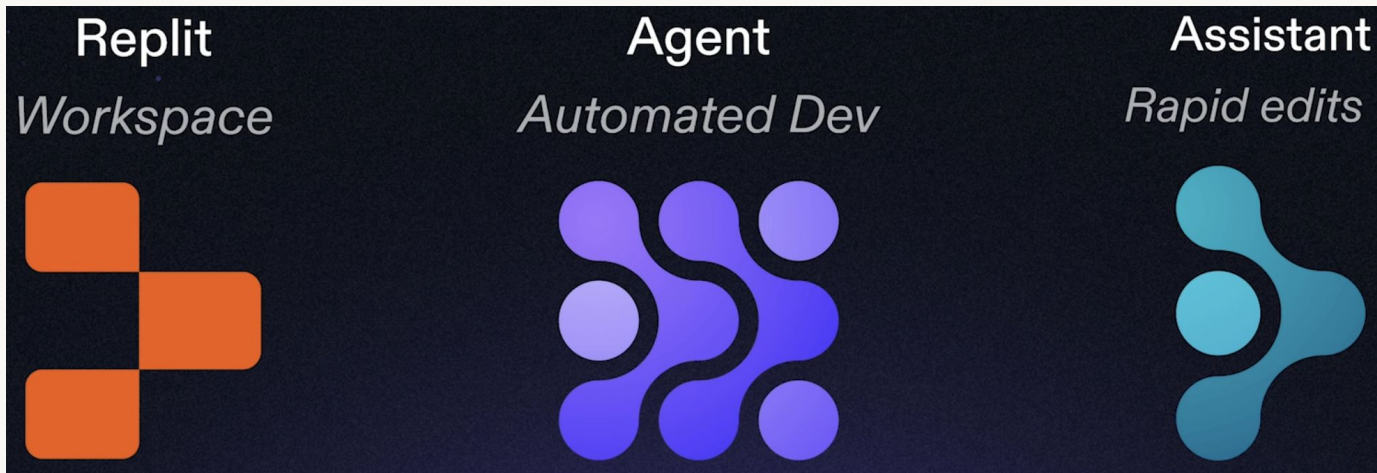
Be Patient

- Build an understanding of the app and its components
 - Replit Agent explains the rationale behind each action including the thought process and changes made to the code
 - Read the output of Replit Agent to learn everything about the app
- Review suggestions before accepting them
- Be patient while debugging – allow the agent time to debug w/ diff strategies

Vibe Coding

- Relying on AI agents or tools to do most of the heavy-lifting on the code
- Large jump in productivity
- Can take from ideation to application more quickly
- Limited by developer experience today – tools, techniques, and environments familiar to the developers
- Has its gotchas – some best practices to keep in mind to work around those and produce better results

Replit



- Use Agent to scaffold out a project and get things to an MVP state
- Flip to Assistant to dial in additional functionality and build out the rest of the project

Replit Workspace

- Zero Setup / No installation: entirely in one browser tab and handles packages and languages
- Includes AI tools for every step of the way
- Has managed stack to build full-stack applications (databases, object storage, authentication)
- Deploy apps in a few clicks

Replit Agent

- Build and configure entire project from scratch
- Generate and build a plan for your project
- Make complex multi-step changes to your code

Replit Assistant

- Chat with AI about any topic
- Request one-off edits to code
- Ask questions about your project

5 Skills in Vibe Coding

- Thinking
- Framework
- Checkpoints
- Debugging
- Context

Thinking

Breaking Thinking into Hierarchies for a game of Chess

- Logical
 - What is the game?
- Analytical
 - How do I play the game?
- Computational
 - What are the patterns behind the game – this helps us to program the computer to enforce the rules of chess
- Procedural
 - How do I excel at the game – programming a computer to play competitive chess – boundaries, edge cases to think about – translate to code/prompt

Framework

- Understand how to do that thing and orient to your problem space
- What frameworks
 - Allow to do that thing, e.g., ask AI to come up with REACT frameworks that help me with drag and drop application
 - Work best with LLMs
- Understand the things the AI is building with via question and iteration loop

Checkpoints

- Things break while building – with or without AI but happens more with AI
- Use version control (or checkpoints in Replit) – “ON” for all projects by default to minimize things breaking
- Chunk up builds, break these down into logical steps, and move quickly in short sprints
- Utilize the checkpoints if anything breaks and rollback

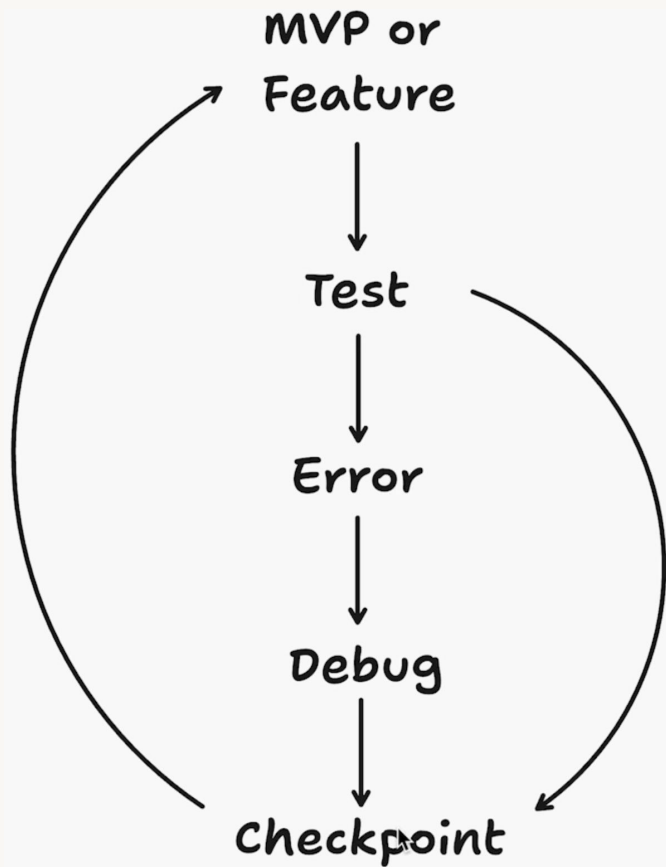
Debugging

- Things tend to break and figuring out why is the process of debugging
- Best debugging is methodical and thorough
- Goals
 - Understand how the app works
 - Understand where the error is
- How can you get to the root cause of this problem?
- How can we tell the agent what's wrong so that it understands and fixes our problem – this is where context comes into play

Context

- Context window: amount of tokens an LLM can process at a given time
- Context can be:
 - Prompt we provide to the LLM
 - Documentation
 - Images
 - Errors
 - Details about your app / environment / preferences
- Since LLMs have a knowledge cut-off, we might have to provide additional context, especially while doing something novel like working with a new package
- Start new chats for new features if the chat history becomes very long

Getting to an MVP

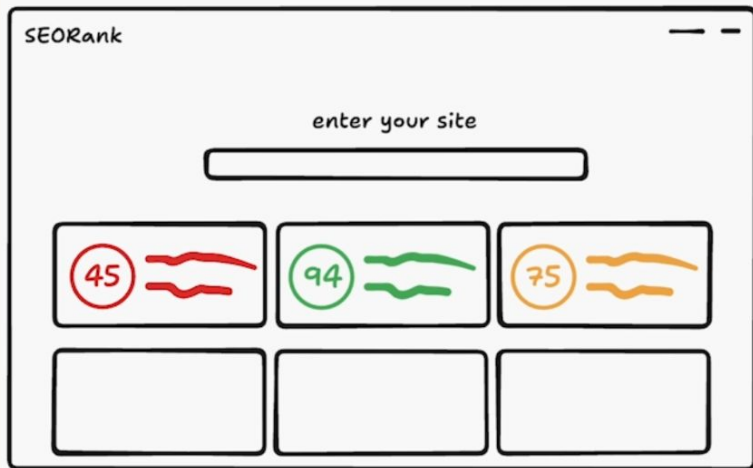


- Give the Agent only the information that's relevant to the MVP
- Start small and work up to something full-featured
- Provide foundational context and important details while crafting the first prompt
- Implement new features by providing context relevant to the new feature. Provide frameworks and documentation w/ explicit implementation details. Example: adding persistent DB storage instead of in-memory post initial set of features are working
- Make incremental changes (checkpoints) and rollback if necessary
- Debug errors. Figure out what's wrong and provide context to the Agent to resolve

Vibe Coding 101 with Replit

Planning and Building SEO Analyzer

Create Wireframe and PRD



An interactive tool that allows you to:

- Enter any website URL
- View a visual representation of the site's load times and tags
- Make sure what we're building is properly configured
- Check if site is properly implemented
- Stores recently analyzed sites

Key skills:

- Prompting
- Processing and analyzing HTML content
- Creating intuitive data visualizations
- Error handling and edge cases
- Storage

Create Prompt

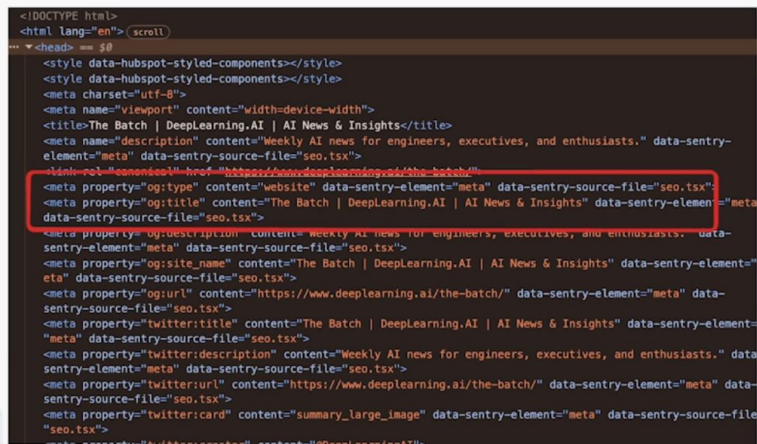
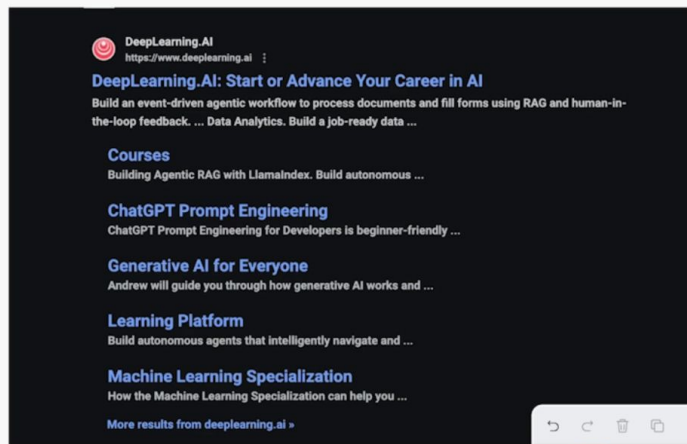
→ The Prompt

domain specific knowledge

Help me create an interactive app that displays the SEO (meta) tags for any website in an interactive and visual way to check that they're properly implemented.

The app should fetch the HTML for a site, then provide feedback on SEO tags in accordance with best practices for SEO optimization. The app should give google and Social media previews.

SEO tags are embedded in the HTML of a site. Anyone can access them.



Create MVP w/ Replit Agent

The screenshot displays the Replit Agent interface for creating an MVP. The top bar shows the project name "SeoTagInspector" and a "Run" button. The left sidebar contains a file explorer with folders like ".git", "client", "server", "shared", and "vite.config.ts". The main chat area shows a conversation with the agent, where the user asks for help creating an interactive app for displaying and analyzing SEO meta tags. The agent responds with a plan and a list of files to be created. A preview window on the right shows the "SEO Meta Tag Analyzer" app, which includes a form to enter a website URL. A bottom panel shows the "Creating file" process, with a code editor displaying the content of "server/storage.ts".

Agent Chat:

seo tag analyzer web app development guide

Help me create an interactive app that displays the SEO (meta) tags for any website in an interactive and visual way to check that they're properly implemented.

The app should fetch the HTML for a site, then provide feedback on SEO tags in accordance with best practices for SEO optimization.

The app should give google and Social media previews.

4 minutes ago

I'll help you create an interactive app for displaying and analyzing SEO meta tags from websites. Let me plan this out for you.

App plan

- Build the initial prototype
 - shadcn/ui

✓ matt approved the plan

Checkpoint made 3 minutes ago • c23f6586 Free

Checkpoint after starting plan

Rollback to here

Created client/index.html

Created shared/schema.ts

Creating server/storage.ts Pause

Message Agent...

Preview:

SEO Meta Tag Analyzer

Analyze and visualize SEO tags to improve your website's optimization

Enter website URL

<https://example.com>

Creating file:

```
server/storage.ts

type InsertUser,
seoAnalyses,
type SeoAnalysis,
type InsertSeoAnalysis
} from "@shared/schema";

// modify the interface with any CRUD methods...
// you might need...

export interface IStorage {
  getUser(id: number): Promise<User | undefined>;
  getUserByUsername(username: string): Promise<User | undefined>;
}
```

Vibe Coding 101 with Replit

Implementing SEO Analysis Features

Feature Expansion w/ Agent

The image displays a dual-pane interface for an AI agent named 'SeoTagInspector'. The left pane shows the agent's instructions and session history, while the right pane shows a live preview of the application being developed.

Left Pane (Agent Interface):

- Agent v2 early access** (Thinking...)
- Make the application more visual:**
 - Create summaries for each category of meta tag that you will display visually to the user, similar to the overall score
 - Make the app overall more visual and allow me to get a summary of the SEO results at a glance
 - Overall, let's make the app more visual and user friendly for folks that might be new to SEO so we can get a high-level view of how well our page is implemented
- Do not remove any functionality, just make it easier to see a high level summary or drill down into the details**
- Checkpoint made just now • e7c6957b** (Free)
- New agent session start** (Rollback to here)
- Thinking...** (Pause)
- Message Agent...**

Right Pane (Preview):

- SEO Meta Tag Analyzer**
- Analyze and visualize SEO tags to improve your website's optimization
- Enter website URL**
-
- Analyze**

Feature Expansion w/ Assistant

The screenshot displays the SeoTagInspector application interface, which is split into two main sections: a chat interface on the left and an SEO analysis results page on the right.

Left Panel (Chat Interface):

- Header:** "SeoTagInspector" with a search icon and a "Running" status indicator.
- Chat Area:**
 - User Message:** "Can you make the website entry form such that https:// is automatically populated and the user doesn't have to enter any other info"
 - Assistant Response:** "I'll help you modify the URLForm component to automatically populate 'https://' and simplify the input process."
 - Code Diff:** Shows changes to `client/src/components/URLForm.tsx` with a green "+14" and red "-9" indicating line additions and deletions.
 - Checkpoints:** A "Checkpoint made" message with a timestamp "just now" and a hash "e605dee7".
- File Explorer:** Lists files in the `.replit` directory, including `client/index.html`, `client/src/components/ResultsOverview.tsx`, `client/src/components/SEOInsights.tsx`, `client/src/components/URLForm.tsx`, and `client/src/pages/Home.tsx`.

Right Panel (SEO Analysis Results):

- Header:** "SEO Meta Tag Analyzer" with a subtitle "Analyze and visualize SEO tags to improve your website's optimization".
- Input Field:** "Enter website URL" with a text input containing "deeplearning.ai" and an "Analyze" button.
- SEO Analysis Results:**
 - Score:** 83 (Overall SEO Score, Very Good).
 - Summary:** "Scored 4 passed, 2 warnings, 0 failed".
 - Check Distribution:** A progress bar showing 67% Passed (green), 33% Warnings (yellow), and 0% Failed (red).
- SEO Health Summary:**
 - Passed Checks:** 4 (green checkmark).
 - Warnings:** 2 (yellow warning icon).
 - Failed Checks:** 0 (red X icon).
- Footer:** "Quick SEO Insights" and "Website analyzed successfully SEO analysis completed".

Deploy the App

The screenshot displays the Vercel IDE interface. On the left, a chat window with an AI assistant is open. The assistant has been prompted to fix an overlapping globe icon by removing it, as the user now has a `https://` prefix. The assistant has responded by removing the overlapping globe icon from the file `client/src/components/URLForm.tsx`. The chat history shows the assistant's previous response and the user's prompt. The right panel shows the 'Autoscale machine configuration' page, which details the machine power and the maximum number of machines for the deployment.

Autoscale machine configuration

Autoscale apps spin up enough machines to meet demand at any point in time. You are billed for the total number of compute units across all machines.

Machine power Edit

The power level for each machine that spins up upon any request

4 vCPUs	72 compute units/sec
8 GiB RAM	16 compute units/sec
Total per machine	88 compute units/sec

Max number of machines

The most machines that should ever spin up to handle your highest traffic. More machines means a faster experience for your users.

3 machines 264 compute units/s at max traffic

[Approve and configure build settings](#)

Your deployed app will utilize any available monthly free allotment before charging for additional resource usage. [Learn more](#)

Deploy the App

The screenshot displays the Replit IDE interface. On the left, a chat window shows a conversation with an AI assistant. The assistant has provided two suggestions to fix an overlapping globe icon by removing it, as the new https:// prefix serves a similar purpose. The first suggestion is for the file `client/src/components/URLForm.tsx` with a +2/-1 diff. The second suggestion is for the same file with a +1/-2 diff. A 'Checkpoint made' message is also visible at the bottom of the chat.

On the right, the 'Deployments' tab is active, showing a progress bar for a deployment started 1 minute ago by Matt. The progress bar indicates that the 'Provision', 'Build', and 'Bundle' steps are complete, while the 'Promote' step is in progress. Below the progress bar, a log of npm notices is displayed, detailing the deployment process:

```
npm notice
2025-03-19T18:54:12Z info: Pushing Repl layer...
2025-03-19T18:54:12Z info: Pushing hosting layer...
2025-03-19T18:54:12Z info: Nix layers for this Repl are uncached.
2025-03-19T18:54:12Z info: Pushing nix-0 layer...
2025-03-19T18:54:12Z info: Pushing pid1 binary layer...
2025-03-19T18:54:12Z info: Created hosting layer
2025-03-19T18:54:14Z info: Created pid1 binary layer
2025-03-19T18:54:28Z info: Created Repl layer
2025-03-19T18:54:32Z info: Created nix-0 layer
2025-03-19T18:54:32Z info: uploading image for cached layer
2025-03-19T18:54:33Z info: pushed cache image manifest
2025-03-19T18:54:33Z info: pushed cache soci index manifest
2025-03-19T18:54:34Z info: uploading cached layer
2025-03-19T18:54:34Z info: successfully uploaded cached layer
2025-03-19T18:54:34Z info: Pushed image manifest
2025-03-19T18:54:35Z info: Pushed soci index manifest
2025-03-19T18:54:39Z info: Creating Autoscale service
```

Vibe Coding 101 with Replit

Planning and Building a Voting App

Create Wireframe and PRD

Lesson 4: Building a head-to-head voting app

nps rank

Which park would you rather visit?

**Great Smoky Mountains**
trending rank #1
Edging open ridge of forest straddles the border between North Carolina and Tennessee in Great Smoky Mountains National Park...

VS

**Bryce Canyon**
trending rank #5
Bryce Canyon National Park, a spectacular reserve in southern Utah, is known for the crimson-colored hoodoos, which are spire-shaped rock formations...

rankings	score	change
1  Grand Canyon	1605	↑ 2
2  Joshua Tree	1538	↓ 1
3  Great Smoky Mountains	1531	— 0
4  Bryce Canyon	1526	↑ 3

A head-to-head ranking app:

- Vote between two US National Parks
- Parks are ranked according to an ELO system (what they use in chess)
 - Used for relative rankings in zero-sum games
- Rankings are displayed in real-time
- Parks and Rankings stored in DB
- Display overall rankings and recent votes

Key skills:

- Prompting
- Persistent data
- Deployments
- Debugging
- Error handling and edge cases

Th

Help me
Parks.

Create Prompt

↪ The Prompt

Help me build an interactive app for voting and ranking the best National Parks.

The app should allow users to vote on parks head to head, then calculate a ranking for the parks based on the chess ELO system. The app should prominently display the matchup along with overall rankings and recent votes.

} Specific use of framework (ELO system)

What about data?

Create MVP w/ Replit Agent

Create a new App

Create with Replit Agent

Choose a Template

Import from GitHub

 en.wikipedia.org/wiki/List_o...



Take screenshot

[Jump to content]
(https://en.wikipedia.org/wiki/List_of_national_parks_of_the_United_States#bodyContent) [[This is a featured list. Click here for more information.]
(<https://upload.wikimedia.org/wikiped>



Help me build an interactive app for voting and ranking the best National Parks.

The app should allow users to vote on parks head to head, then calculate a ranking for the parks based on the chess ELO system.

The app should prominently display the matchup along with overall rankings and recent votes.

https://en.wikipedia.org/wiki/List_of_national_parks_of_the_United_States

 App type: Web app (Python) ▾ 



 Start building

 Book scanner

 AI chat

 Waitlist website

Create MVP w/ Replit Agent

Agent x Assistant +

New chat

National Parks Voting App ELO Ranking System



Help me build an interactive app for voting and ranking the best National Parks.

The app should allow users to vote on parks head to head, then calculate a ranking for the parks based on the chess ELO system.

The app should prominently display the matchup along with overall rankings and recent votes.

https://en.wikipedia.org/wiki/List_of_national_parks_of_the_United_States

46 minutes ago

I understand you want to build an interactive app for voting and ranking the best National Parks using a Flask with Vanilla JavaScript stack. Let me analyze your requirements and propose a plan for this project.

App plan

- Build the initial prototype

App type Web app (Python)

Timeline

- Visual preview (~2 min.)
- First app version (~10 min.)

Get notified when your app is ready

✓ visheshharya87 approved the plan

Scroll to latest

This chat has ended

Start a new Agent chat

Preview x Console Database Code Search +

[...]replit.dev /

Ready to share with the world? Deploy

 **NPS Rank**

Vote Rankings

Which park would you rather visit?



Acadia

Maine

Acadia features the tallest mountain on the Atlantic coast of the United States, granite peaks, ocean shoreline, woodlands, and lakes.

trending

VS



Arches

Utah

This site features more than 2,000 natural sandstone arches, including the Delicate Arch.

trending

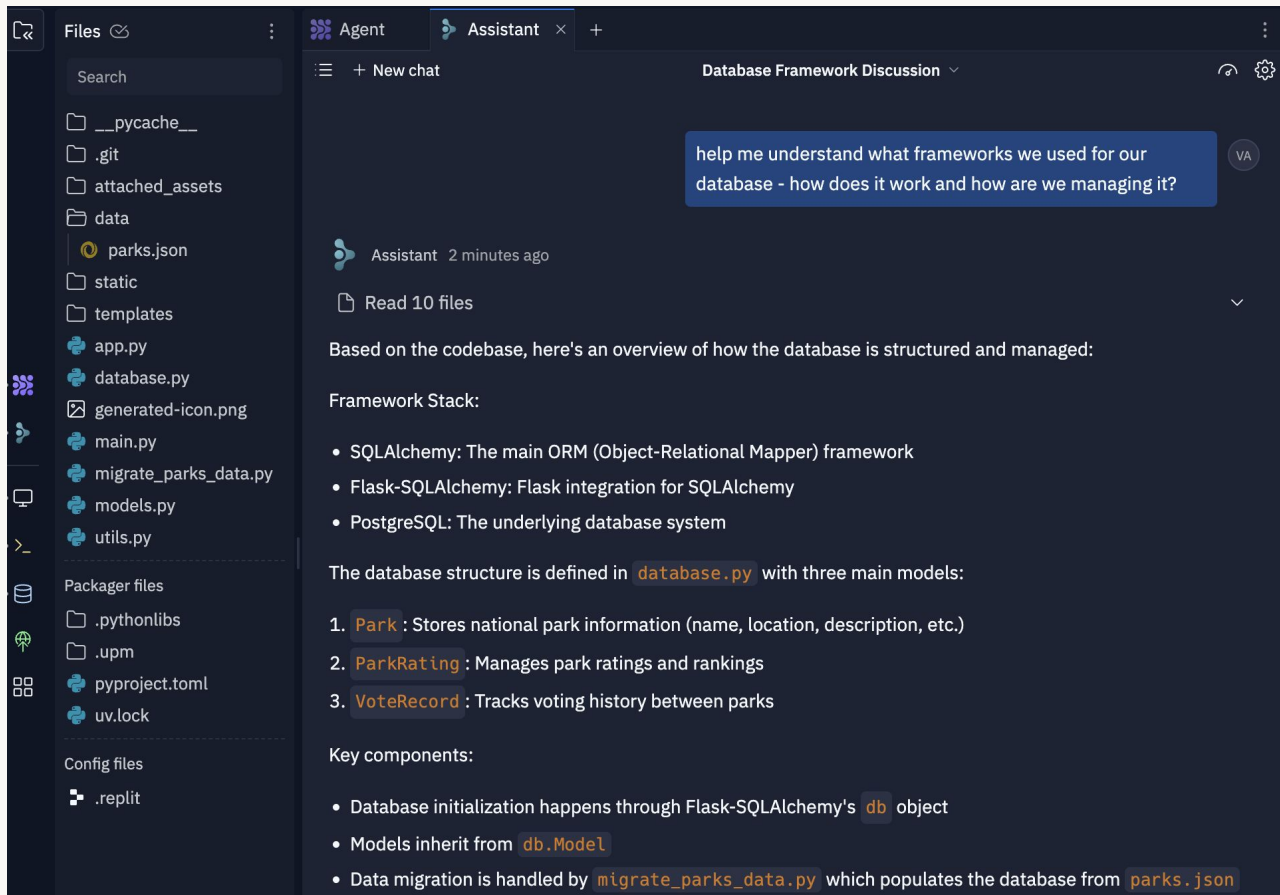
Rankings

Rank	Park	Score	Change
1	 Capitol Reef Utah	1544	— 0

Vibe Coding 101 with Replit

Enhance the Parks Voting App

Understanding Project w/ Assistant



The screenshot displays a code editor interface with a file explorer on the left and a chat window on the right. The file explorer shows a project structure with files like `__pycache__`, `.git`, `attached_assets`, `data`, `parks.json`, `static`, `templates`, `app.py`, `database.py`, `generated-icon.png`, `main.py`, `migrate_parks_data.py`, `models.py`, `utils.py`, `.pythonlibs`, `.upm`, `pyproject.toml`, `uv.lock`, and `.replit`.

The chat window, titled "Database Framework Discussion", shows a user asking: "help me understand what frameworks we used for our database - how does it work and how are we managing it?". The assistant responds with the following information:

Based on the codebase, here's an overview of how the database is structured and managed:

Framework Stack:

- SQLAlchemy: The main ORM (Object-Relational Mapper) framework
- Flask-SQLAlchemy: Flask integration for SQLAlchemy
- PostgreSQL: The underlying database system

The database structure is defined in `database.py` with three main models:

1. `Park` : Stores national park information (name, location, description, etc.)
2. `ParkRating` : Manages park ratings and rankings
3. `VoteRecord` : Tracks voting history between parks

Key components:

- Database initialization happens through Flask-SQLAlchemy's `db` object
- Models inherit from `db.Model`
- Data migration is handled by `migrate_parks_data.py` which populates the database from `parks.json`

Deploy the App

Drizzle ORM is being used to manage the

des several benefits:

`ema.ts` defines TypeScript types for all
providing compile-time type checking.
sy to define and modify database schemas.

Copy

```
arks", {
```

```
).notNull(),
```

`server/storage.ts` shows how Drizzle

Copy

```
FROM parks ORDER BY elo DESC
```

```
Park[]> {
```

Autoscale machine configuration

Autoscale apps spin up enough machines to meet demand at any point in time. You are billed for the total number of compute units across all machines.

⚡ Machine power

Edit

The power level for each machine that spins up upon any request

4 vCPUs

72 compute units/sec

8 GiB RAM

16 compute units/sec

Total per machine

88 compute units/sec ⓘ

📖 Max number of machines

The most machines that should ever spin up to handle your highest traffic. More machines means a faster experience for your users.

3 machines

264 compute units/s at max traffic ⓘ

Approve and configure build settings →

Deploy the App

Drizzle ORM is being used to manage the database.

It provides several benefits:

- `drizzle-orm` defines TypeScript types for all database queries, providing compile-time type checking.
- It's easy to define and modify database schemas.

```
const parks = db.parks;

// ...

// server/storage.ts shows how Drizzle ORM is used to manage the database.
```

```
FROM parks ORDER BY elo DESC
Park[]> {
  // ...
  orderBy(desc(parks.elo));
}
```

Type **Autoscale** (4 vCPU / 8 GiB RAM / 3 Max) [Manage](#)

Primary domain ⓘ
.replit.app
✓ Available

Private Deployment  [Teams](#) ☐
Limit deployment access to team members. [Learn more](#)

Build command ⓘ optional

Run command ⓘ

Deployment secrets ⓘ

<>	 SESSION_SECRET			
<>	 DATABASE_URL			
<>	 DATABASE			
<>	 PGHOST			
<>	 PGPORT			

Deploy the App

Deploying Started 1 minute ago by Matt Cancel

Hide logs ^

Provision ✓

Build ✓

Bundle ⌚

Promote ⌚

dist/index.js 37.6kb

⚡ Done in 13ms

npm notice

npm notice New major version of npm available! 10.8.2 -> 11.2.0

npm notice Changelog: <https://github.com/npm/cli/releases/tag/v11.2.0>

npm notice To update run: npm install -g npm@11.2.0

npm notice

2025-03-19T21:17:33Z info: Pushing pid1 binary layer...

2025-03-19T21:17:34Z info: Created pid1 binary layer

2025-03-19T21:17:35Z info: Pushing Repl layer...

2025-03-19T21:17:35Z info: Pushing nix-0 layer...

2025-03-19T21:17:35Z info: Created hosting layer

2025-03-19T21:17:57Z info: Created Repl layer

2025-03-19T21:17:57Z info: Pushing Repl (cache) layer...

2025-03-19T21:17:57Z info: Created Repl (cache) layer



Deploy the App

 Redeploy

Edit commands and secrets

Production

Status ● Matt deployed 5 minutes ago 

Domain <https://park-challenge.replit.app>  

Type Autoscale (4 vCPU / 8 GiB RAM / 3 Max) [Manage](#) [See all usage](#)